

LaoSA: An Approach Based on Fine-tuning PLMs for Lao Sentiment Analysis

Khamtay Kongmanh¹, Quang-Vinh Pham², Quang-Hung Le^{3*}

¹ Department of Information Technology, Quy Nhon University, Vietnam
kongmanh4551050274@st.qnu.edu.vn

² Department of Data Science, TMA Tech Group, Vietnam
pqvinh@tma.com.vn

³ Department of Information Technology, Quy Nhon University, Vietnam
lequanghung@qnu.edu.vn

Abstract

Effective sentiment analysis for the Lao language is hindered by a scarcity of annotated resources and the inherent noise of real-world user feedback, which often features code-mixing and informal script patterns. This study addresses these challenges by introducing a high-quality, filtered benchmark of 25,139 Lao-language reviews collected from popular digital applications. We evaluate several state-of-the-art Pre-trained Language Models (PLMs), including XLM-RoBERTa, mBERT, and a Lao-specific mBERT-Lao, across various adaptation strategies such as Full Fine-tuning and Low-Rank Adaptation (LoRA). Our experimental results demonstrate that XLM-RoBERTa achieves superior performance with 96.64% accuracy and an F1-macro score of 0.9617. We also find that while Lao-oriented pretraining provides significant benefits, LoRA offers a highly efficient performance-cost trade-off, maintaining competitive accuracy with minimal trainable parameters. Our research highlights the robustness of multilingual transfer for low-resource sentiment classification and provides a publicly available dataset and code-mixing benchmark at <https://github.com/KT246/LaosSA>.

Index Terms

Lao Sentiment Classification, Pre-trained Language Models, XLM-RoBERTa, LoRA, Low-resource NLP.

1 Introduction

Sentiment analysis plays an important role in e-commerce and digital-service platforms because user reviews and ratings provide direct evidence of customer satisfaction, service quality, and product perception [1], [2]. For Lao-language platforms, automatic sentiment classification could support faster and more scalable analysis of user feedback from app stores, social media, and related digital channels.

The main gap is that Lao sentiment analysis remains under-resourced in both data and tooling. Publicly available annotated corpora, standardized benchmarks, and mature NLP tools for Lao are still limited [3], [4]. The challenge is compounded by the continuous-script nature of Lao, in which word boundaries are not explicitly marked [4], and by the character of real review data, which is often short, noisy, informal, and domain-specific [5]–[7]. As a result, methods that work well for high-resource languages do not transfer cleanly to Lao review classification.

This limitation affects both traditional and pretrained approaches. Survey work on sentiment analysis notes that rule-based systems demand substantial manual effort, while classical machine learning models remain sensitive to feature design and labeled data availability [1], [2]. Generic multilingual PLMs are not guaranteed to perform well either, because their pretraining may underrepresent Lao and may not align closely with Lao tokenization behavior or review-specific language [3]. This motivates a direct comparison between strong multilingual PLMs, a Lao-supporting PLM, and sparse baselines on the same Lao app-review dataset. Our working hypothesis is that stronger multilingual transfer and Lao-oriented pretraining will outperform

sparse baselines, while parameter-efficient tuning will preserve most of the gains at a lower training cost.

In this work, we study Lao sentiment classification using reviews collected from Google Play applications that are actively used in Laos. Although the initial motivation comes from e-commerce sentiment analysis, the final dataset covers a broader Lao digital-service setting, including banking, social media, and other everyday mobile applications. The main contributions are as follows:

- We construct and analyze a Lao sentiment dataset from real mobile application reviews used in Laos, and we document the filtering steps that produce the final train and validation splits.
- We benchmark three pre-trained language models, namely XLM-RoBERTa, mBERT, and mBERT-Lao, under Full Fine-tuning, LoRA, and 3-Fold Cross-Validation settings, and we compare them against Logistic Regression, SVM, and Decision Tree baselines.
- We report predictive performance, error patterns, and training efficiency so that model quality and training cost can be examined together.

The remainder of this paper is organized as follows. Section 2 reviews related work on sentiment analysis, low-resource language processing, and pre-trained language models. Section 3 describes dataset construction and analysis, including data collection, annotation, preprocessing, and descriptive statistics. Section 4 then formulates the task, defines the evaluation protocol, and presents the modeling framework and fine-tuning strategies. Section 5 reports the experimental setup, results, and discussion. Finally, Section 6 concludes the paper and outlines future directions.

2 Related Work

Sentiment analysis is widely used to summarize user opinions from reviews, ratings, and social media posts, so it remains relevant to both e-commerce and broader digital-service platforms. Recent surveys show that review-level polarity classification remains the most common setting [1], [2]. A related but distinct line of work studies aspect-level sentiment analysis, where the goal is to identify which product or service attributes trigger positive or negative feedback [8]–[12]. In the present study, we focus on review-level polarity because the Lao app-review dataset contains one short opinion per entry and does not include aspect annotations.

Domain effects are important because sentiment expressions vary across application types and user communities. Studies on Tamil and Tulu sentiment classification report that code-mixing, informal spelling, domain variation, and dataset imbalance can noticeably reduce robustness, especially for traditional machine learning models [2], [5]–[7]. Similar issues appear in Lao app reviews, where many comments are short, colloquial, and tied to specific application behaviors such as login errors, transaction problems, or service satisfaction. These properties make domain-aware evaluation necessary even when the task is framed as simple review-level sentiment classification.

Low-resource language processing is still constrained by the lack of large corpora, annotated benchmarks, and mature preprocessing tools. For Lao, this limitation has been noted in work on word segmentation, text classification, and broader resource construction [3], [4]. Because Lao is written as a continuous script, preprocessing itself can become a source of error: tokenization and boundary detection are less straightforward than in whitespace-delimited languages [4]. This challenge is directly relevant to the current dataset, where many user reviews are very short and noisy.

To address these limitations, recent work on under-resourced languages relies heavily on transfer learning. Recent surveys of low-resource sentiment analysis highlight multilingual PLMs, few-shot transfer, and cross-lingual learning as common strategies [13], while broader multilingual NLP studies explain why transfer-based evaluation remains important in low-resource settings [14], [15]. At a broader regional level, cross-lingual representation alignment for ASEAN languages suggests that neighboring-language transfer can still provide useful signals for Lao when direct resources are scarce [16]. For Lao, such methods are useful because labeled data remains limited, although their effectiveness can still vary with language coverage

and domain mismatch. This motivates testing both multilingual models and a Lao-supporting model on the same review dataset.

Pre-trained language models are now the dominant backbone for text classification because they provide contextualized representations that can be adapted across tasks [15], [17]. For Lao, the distinction between monolingual and multilingual PLMs is particularly important [3]. A Lao-oriented model can better reflect local tokenization behavior and lexical patterns, whereas multilingual models benefit from broader pretraining but may devote less capacity to low-resource languages [3], [14], [15].

Recent low-resource sentiment studies still report clear gains from transformer-based models, but they also show that performance depends on language fit, adaptation strategy, and dataset characteristics [5]–[7]. This is why our experiments compare XLM-RoBERTa and mBERT with mBERT-Lao, and also keep strong sparse baselines in the benchmark.

3 Dataset Construction and Analysis

This section describes how the Lao review benchmark was collected, annotated, filtered, and summarized before model training.

3.1. Data Collection

The reviews used in this study were collected from Google Play applications that are widely used in Laos. The corpus includes comments from banking apps, social media apps, and other digital-service applications rather than from a single e-commerce marketplace. This choice was practical: these applications produced enough Lao user feedback to build a usable sentiment dataset, and they reflect the short mobile reviews that a deployed Lao sentiment system would need to handle.

Each record contains two fields, namely the review text and a binary sentiment label. The final benchmark also retains short code-mixed reviews in which Lao words appear together with English interface terms such as *app*, *bug*, or *OK*, provided that the overall sentiment remains interpretable. This choice keeps the released benchmark closer to real mobile feedback in Laos and directly corresponds to the code-mixing setting referenced in the released resources.

3.2. Annotation Scheme

The raw reviews were manually annotated with binary sentiment labels. Each review was assigned a single label based on its overall polarity: label 1 for positive sentiment and label 0 for negative sentiment. The annotation process focused on the dominant sentiment expressed in each review, including explicit praise, complaints, dissatisfaction, or statements about overall user experience.

To improve label consistency, the annotation followed a simple set of practical guidelines. Reviews expressing clearly favorable opinions toward an application, service quality, or user experience were marked as positive, whereas reviews expressing frustration, malfunction, dissatisfaction, or criticism were marked as negative. During dataset refinement, noisy, invalid, or suspicious samples were revisited through the cleaning pipeline described below so that only reliable sentiment instances were retained in the final corpus.

3.3. Data Preprocessing and Cleaning Pipeline

Before final training, the raw reviews were cleaned and filtered in several stages. After manual labeling, we removed noisy content such as repeated characters, emojis, invalid symbols, and strings that did not form meaningful Lao text. Particular attention was paid to Lao-specific character errors, especially cases in which vowels appeared without consonants or in which character combinations were visually present but linguistically invalid.

To assess label quality more efficiently, we trained a lightweight screening model and compared its predictions with the manual annotations. Samples that repeatedly agreed with their labels were retained, whereas mismatched cases were rechecked and many were removed. This step served as a practical filter for suspicious or low-quality reviews before the final benchmark experiments.

We also used a separate 5-fold cross-validation auditing step for difficult cases during dataset cleaning. This step was part of data filtering only and is distinct from the 3-fold cross-validation used later for model evaluation. Reviews that were repeatedly misclassified across folds were treated as unstable, and many of them contained invalid Lao character patterns or meaningless text fragments.

After this filtering process, the remaining reviews formed the final dataset used for training and validation.

3.4. Dataset Statistics

Table I summarizes the final train and validation splits after cleaning and filtering. The two partitions remain closely matched in class balance and contain no missing text or labels. The reviews are short on average, with only a small number of much longer comments, which is consistent with the brief and noisy style of app-store feedback.

TABLE I
FINAL TRAIN AND VALIDATION SPLIT STATISTICS FOR THE LAO REVIEW BENCHMARK AFTER FILTERING AND PREPROCESSING.

Statistic	Train	Validation
Rows	20,111	5,028
Columns	2	2
Empty text	0	0
Empty label	0	0
Unique text	20,095	5,028
Duplicated text within split	16	0
Negative reviews (Label = 0)	6,312 (31.39%)	1,578 (31.38%)
Positive reviews (Label = 1)	13,799 (68.61%)	3,450 (68.62%)
Minimum text length (characters)	2	3
Average text length (characters)	29.41	30.29
Maximum text length (characters)	618	506
Minimum word count	1	1
Average word count	1.52	1.52
Maximum word count	66	77

4 Methodology

This section formalizes the benchmark task, describes the compared PLMs and adaptation strategies, and summarizes the shared training pipeline used in the experiments.

4.1. Problem Definition

We formulate Lao sentiment classification as a supervised binary text classification task over cleaned Lao reviews. Let the benchmark dataset be defined as

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N, \quad (1)$$

where each review $x_i = (w_1, w_2, \dots, w_n)$ is represented by the token sequence produced by a given tokenizer, and each label satisfies $y_i \in \{0, 1\}$, with 0 denoting negative sentiment and 1 denoting positive sentiment. Each review receives a single sentence-level label corresponding to its dominant polarity, even when the text remains short, informal, or noisy.

The task is to learn a classifier f_ψ that maps a Lao review to one of the two sentiment classes. For each input x , a model estimates a posterior distribution over the label space and predicts the dominant polarity according to

$$\hat{y} = f_\psi(x) = \arg \max_{c \in \{0, 1\}} p_\psi(y = c \mid x), \quad (2)$$

where ψ denotes the parameters of either a classical classifier or an adapted PLM-based classifier, and c indexes the candidate sentiment class. This formulation keeps the task definition model-agnostic so that sparse baselines and PLM-based systems can be compared under the same prediction rule.

The benchmark is defined under several practical assumptions. Each review is treated as an independent sample without conversational context or user metadata. The experiments focus on document- or sentence-level polarity classification rather than aspect-level sentiment extraction, and all systems operate directly on the cleaned review text rather than on hand-crafted linguistic features. These assumptions keep the comparison centered on representation quality and adaptation strategy rather than on feature design or external metadata.

The evaluation protocol follows the final filtered split described in Section 3.4. For the main benchmark, models are trained on 20,111 training samples and evaluated on a fixed validation set of 5,028 samples. In addition, a separate 3-fold cross-validation setting is used as a robustness check on the training partition. Performance is reported using Accuracy, Precision-macro, Recall-macro, and F1-macro, with F1-macro treated as the primary comparison metric because of the moderate class imbalance between positive and negative reviews.

4.2. Pre-trained Language Models

We evaluate three PLMs: XLM-RoBERTa, mBERT, and mBERT-Lao. XLM-RoBERTa and mBERT serve as multilingual references for cross-lingual transfer [14], [15], [17], while mBERT-Lao provides a Lao-supporting alternative [3]. Table II summarizes the main properties of the three models under the benchmark-facing names used throughout this paper. All PLM runs share the same preprocessing and evaluation protocol, and the LoRA setting attaches adapters to the query, key, and value projections.

TABLE II
PRE-TRAINING AND ARCHITECTURAL COMPARISON OF THE PLMS USED IN THE BENCHMARK.

Aspect	XLM-RoBERTa	mBERT	mBERT-Lao
Language scope	Multilingual (100 languages)	Multilingual (104 languages)	Lao-supporting
Pre-training data	CommonCrawl multilingual corpus (~2.5 TB)	Multilingual Wikipedia corpus	Lao-oriented RoBERTa-style pretraining
Tokenizer	SentencePiece, ~250k subwords	WordPiece, ~119k vocabulary	RoBERTa tokenizer, ~50k vocabulary
Architecture (base)	12 layers, 768 hidden, 12 heads	12 layers, 768 hidden, 12 heads	12 layers, 768 hidden, 12 heads
Role in this benchmark	Strong multilingual reference	Strong multilingual reference	Lao-supporting comparison model

4.3. Fine-Tuning Strategies

Each PLM is trained with either Full Fine-tuning or LoRA. Full Fine-tuning updates the backbone and classification head end to end. LoRA instead freezes the pretrained weights and learns low-rank updates in selected attention projections [18]–[20]. We also report 3-fold cross-validation as a robustness check; in that setting, a separate model is trained on each fold and the final score is computed from the concatenated out-of-fold predictions.

Let θ denote the backbone parameters and ϕ the classification head. Full Fine-tuning solves

$$(\theta^*, \phi^*) = \arg \min_{\theta, \phi} \mathcal{L}_{\text{train}}(\theta, \phi). \quad (3)$$

For LoRA, the pretrained matrix is decomposed as $W = W_0 + BA$, where W_0 is frozen and $A \in \mathbb{R}^{r \times d}$ and $B \in \mathbb{R}^{d \times r}$ are trainable. The optimization becomes

$$(A^*, B^*, \phi^*) = \arg \min_{A, B, \phi} \mathcal{L}_{\text{train}}(W_0 + BA, \phi). \quad (4)$$

For cross-validation, out-of-fold predictions from all K folds are concatenated and scored jointly. The reported score is therefore

$$\text{Score}_{\text{CV}} = \text{Score} \left(\bigcup_{k=1}^K \hat{D}_{\text{val}}^{(k)} \right), \quad K = 3, \quad (5)$$

where $\hat{D}_{\text{val}}^{(k)}$ denotes the predictions produced by the best checkpoint of fold k on its held-out partition.

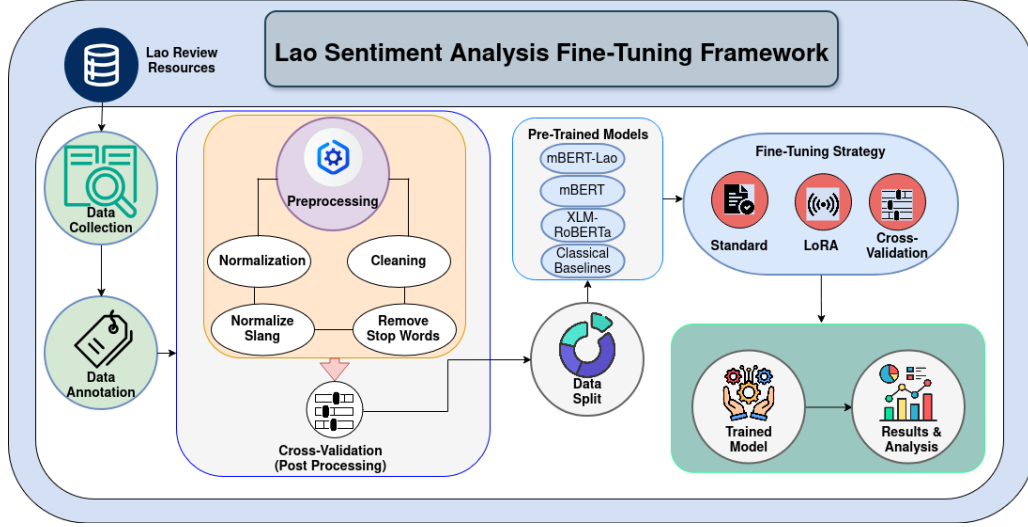


Fig. 1. Overall methodology framework for Lao sentiment classification, including dataset preparation, model selection, and fine-tuning strategies.

Figure 1 summarizes the benchmark in three stages. The workflow begins with Google Play review collection, binary annotation, normalization, cleaning, and the post-processing audit used to construct the final split. It then trains XLM-RoBERTa, mBERT, mBERT-Lao, and TF-IDF baselines under Full Fine-tuning, LoRA, and 3-fold cross-validation. Finally, the pipeline exports checkpoints, validation predictions, standard metrics, timing summaries, and false-positive/false-negative artifacts for the later analysis sections.

4.4. Training Objective

PLM runs minimize weighted cross-entropy loss. Class weights compensate for the moderate label imbalance in the final dataset. For a training set of N samples and two sentiment classes, the objective is

$$\mathcal{L} = - \sum_{i=1}^N \sum_{c=1}^2 w_c y_{i,c} \log \hat{y}_{i,c}, \quad (6)$$

where w_c is the weight for class c , $y_{i,c}$ is the one-hot target, and $\hat{y}_{i,c}$ is the predicted probability for class c .

We train the PLMs with standard pretrained-model tooling and fine-tuning practice [15], [17], using a learning rate of $1e^{-5}$, weight decay 0.01, warmup ratio 0.1, and maximum sequence length 128. For Full Fine-tuning and LoRA, the best checkpoint is selected by validation loss. The classical baselines use the same split and train TF-IDF character n -gram features from 3 to 5 with `min_df=2` and at most 50,000 features before classifier fitting.

Algorithm 1 summarizes the unified benchmark pipeline used throughout the experiments. It reflects the fixed-split baseline runs, the PLM-based fixed-split runs, and the separate 3-fold cross-validation setting reported later in Section 5.

Algorithm 1 Unified training and evaluation pipeline for Lao sentiment classification

Require: fixed-split training set D_{train} , validation set D_{val} , PLM set $\mathcal{M}$, baseline set $\mathcal{B}$, strategies $\mathcal{S} = \{\text{FT}, \text{LoRA}\}$

Ensure: prediction files, metrics, timing statistics, and error-analysis artifacts

```
1: for all  $b \in \mathcal{B}$  do
2:   compute TF-IDF features  $\Phi(x)$  for  $x \in D_{\text{train}}$ 
3:    $\psi_b \leftarrow \arg \min_{\psi} \mathcal{L}_{\text{train}}(\psi, \Phi(D_{\text{train}}))$ 
4:    $\hat{y} \leftarrow f_{\psi_b}(\Phi(x))$  for  $x \in D_{\text{val}}$ , evaluate metrics
5: end for
6: for all  $m \in \mathcal{M}$  do
7:   for all  $s \in \mathcal{S}$  do
8:     load tokenizer and pretrained checkpoint for  $m$ 
9:     if  $s = \text{LoRA}$  then
10:      inject LoRA adapters  $BA$  via Eq. (4)
11:     else
12:      enable end-to-end updates via Eq. (3)
13:     end if
14:     minimize  $\mathcal{L}$  on  $D_{\text{train}}$  via Eq. (6)
15:      $\psi^* = \arg \min_{\psi} \mathcal{L}(\psi, D_{\text{val}})$ 
16:     compute  $\hat{y} = f_{\psi^*}(x)$  for  $x \in D_{\text{val}}$ , store metrics
17:   end for
18:   for  $k = 1$  to 3 do
19:     partition  $D_{\text{train}} \rightarrow \{D_{\text{train}}^{(k)}, D_{\text{val}}^{(k)}\}$ 
20:      $\psi^{(k)} \leftarrow \text{optimize}(\mathcal{L}, D_{\text{train}}^{(k)}, m)$ 
21:      $\hat{D}_{\text{val}}^{(k)} \leftarrow \{f_{\psi^{(k)}}(x) \mid x \in D_{\text{val}}^{(k)}\}$ 
22:   end for
23:   compute  $\text{Score}_{\text{CV}}$  using Eq. (5)
24: end for
25: aggregate  $\hat{D}_{\text{val}}$  and metrics to produce final artifacts
```

5 Experiments

This section describes the benchmark configuration and then analyzes the main performance, error, and efficiency findings.

5.1. Experimental Setup

We begin by specifying the compared systems, the evaluation metrics, and the implementation details required to reproduce the benchmark.

5.1.1. Baseline Models

The benchmark combines classical baselines and PLM-based models. The classical group contains Logistic Regression, SVM, and Decision Tree. The PLM group contains XLM-RoBERTa, mBERT, and mBERT-Lao, each evaluated with Full Fine-tuning, LoRA, and 3-Fold Cross-Validation. This setup creates a controlled comparison between sparse models and pretrained transformers on the same Lao review data.

The classical baselines were trained on the same fixed train-validation split used for the fixed-split PLM runs. They use character-level TF-IDF features with an n -gram range of 3-5, $\text{min_df}=2$, and at most 50,000 features. Logistic Regression and SVM were implemented through `SGDClassifier` with `log_loss` and `hinge` objectives, respectively, while Decision Tree used `max_depth=40` and `min_samples_leaf=2`. Neural baselines such as CNNs or BiLSTMs are not included because the current project artifacts do not contain trained runs for those models.

5.1.2. Evaluation Metrics

We report Accuracy, Precision-macro, Recall-macro, and F1-macro, which are standard evaluation measures for text classification and information retrieval tasks [1], [21]. Because the dataset is moderately imbalanced, F1-macro is treated as the main comparison metric, while the remaining metrics are used to show overall correctness and class-level balance.

For binary sentiment classification, Accuracy is defined as

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \quad (7)$$

For each class $c \in \{0, 1\}$, let

$$P_c = \frac{TP_c}{TP_c + FP_c}, \quad R_c = \frac{TP_c}{TP_c + FN_c}. \quad (8)$$

The macro-averaged scores reported in this paper are then

$$\text{Precision}_{\text{macro}} = \frac{1}{C} \sum_{c=1}^C P_c, \quad \text{Recall}_{\text{macro}} = \frac{1}{C} \sum_{c=1}^C R_c, \quad \text{F1}_{\text{macro}} = \frac{1}{C} \sum_{c=1}^C \frac{2P_c R_c}{P_c + R_c}, \quad (9)$$

where c indexes a sentiment class and $C = 2$ for the negative and positive labels. Here, TP , TN , FP , and FN denote true positives, true negatives, false positives, and false negatives, respectively, while TP_c , FP_c , and FN_c are the corresponding class-specific counts used to compute P_c and R_c for class c .

5.1.3. Implementation Details

The experiments use the final filtered Lao review dataset described in Section 3.4. Logistic Regression, SVM, and Decision Tree were trained on the same fixed split as the fixed-split PLM runs, while 3-fold cross-validation was applied only on the training partition. The exact hyperparameters are listed in Table III; the main difference across PLM runs is whether the full backbone is updated or only LoRA adapters are trained.

TABLE III
TRAINING CONFIGURATION USED IN THE EXPERIMENTS.

Component	Configuration
PLM epochs	25
Batch size / eval batch size	16 / 32
Maximum sequence length	128
Learning rate	1×10^{-5}
Weight decay	0.01
Warmup ratio	0.1
Cross-validation	Stratified $K = 3$
LoRA setting	$r = 16$, $\alpha = 32$, dropout = 0.1
Best-model criterion	Validation loss
Baseline features	TF-IDF char n -grams (3-5), max 50,000

TABLE IV
RUNTIME ENVIRONMENT RECORDED IN THE EXPERIMENT ARTIFACTS.

Hardware item	Recorded value
CPU	1 physical core / 2 total cores
RAM	12.67 GB
GPU	Tesla T4
GPU memory	14.56 GB
Recorded hardware scope	12 experiment folders

5.2. Results and Discussion

We now examine the main performance pattern and use error and efficiency analyses to explain the observed trade-offs.

5.2.1. Overall Performance

Taken together, Table V and Figure 2 indicate that backbone suitability matters more than the choice between Full Fine-tuning and LoRA in this benchmark. This interpretation is consistent with low-resource sentiment studies that attribute transfer quality mainly to language fit and data characteristics rather than to simply updating more parameters [5], [6], [13]. The benchmark also shows that sparse baselines remain meaningful for Lao app reviews, suggesting that the task still contains strong lexical sentiment cues even when pretrained encoders are available.

TABLE V
OVERALL PERFORMANCE GROUPED BY MODEL FAMILY AND TRAINING STRATEGY.

Model Family	Training Strategy	Acc.	F1	Prec.	Rec.
XLM-RoBERTa	Full Fine-tuning	0.9664	0.9617	0.9544	0.9702
	LoRA	0.9642	0.9589	0.9539	0.9645
	Cross-Validation	0.9648	0.9597	0.9541	0.9659
mBERT	Full Fine-tuning	0.6969	0.6057	0.6336	0.6012
	LoRA	0.6901	0.6048	0.6253	0.6004
	Cross-Validation	0.6967	0.6137	0.6350	0.6084
mBERT-Lao	Full Fine-tuning	0.9407	0.9328	0.9241	0.9437
	LoRA	0.9326	0.9235	0.9154	0.9335
	Cross-Validation	0.9375	0.9285	0.9228	0.9350
Classical baselines	Logistic Regression	0.9451	0.9370	0.9322	0.9425
	SVM	0.9505	0.9436	0.9363	0.9522
	Decision Tree	0.8878	0.8721	0.8662	0.8792

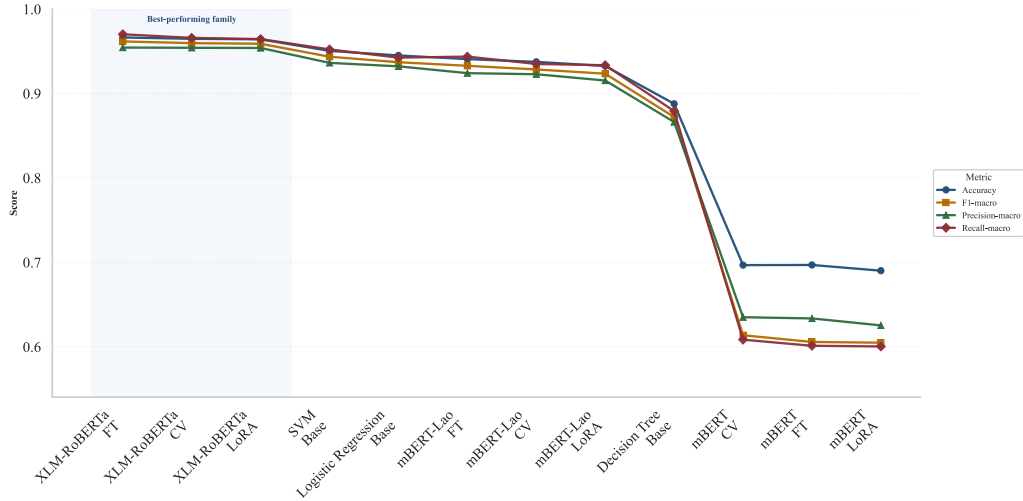


Fig. 2. Metric comparison across PLM settings and classical baselines, sorted by F1-macro.

5.2.2. Domain Adaptation Analysis

The gap between mBERT-Lao and generic mBERT suggests that Lao support is not a minor implementation detail but a substantive source of performance difference. This interpretation aligns with LaoPLM and related low-resource work, which argue that multilingual pretraining can under-serve Lao when tokenization behavior and corpus coverage are mismatched [3], [13]. At the same time, the competitiveness of SVM and Logistic Regression indicates that many app-review decisions are driven by local lexical cues and short formulaic complaints. In other words, domain adaptation in this dataset appears to depend on both language-aware representations and sensitivity to concise, review-specific expressions.

5.2.3. Error Analysis

Figure 3 helps explain why the model ranking separates so clearly. The strongest systems maintain a more balanced error profile, whereas weaker multilingual transfer is associated with a clear positive-class bias. This matters because the difficult cases are not random; they are concentrated in short complaint-like comments, code-mixed fragments, and mixed-polarity reviews that combine a problem report with a request, update, or partially positive tone. Related low-resource sentiment studies likewise report that short, informal, and mixed-language comments are a recurring source of ambiguity [5], [6], but the present results suggest that these cases

are especially consequential for Lao app reviews, where limited context and informal phrasing amplify ambiguity.

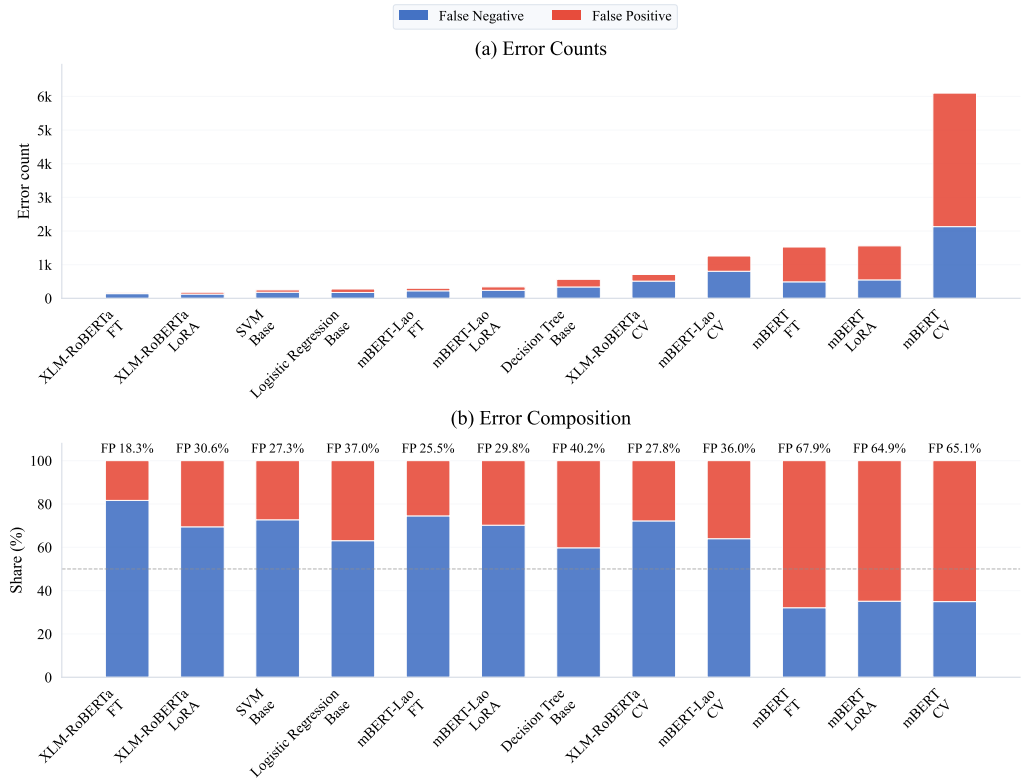


Fig. 3. Error-analysis overview based on the aggregated false-positive and false-negative samples from all experiments. Panel (a) reports raw false-negative and false-positive counts, while panel (b) shows the corresponding error composition percentages for the same experiments.

Table VI illustrates this pattern qualitatively. False positives are usually short complaints whose negative intent is expressed indirectly or through code-mixed wording, whereas false negatives more often contain softened criticism, requests, or mixed cues. The dominant failure mode is therefore semantic ambiguity in short feedback, rather than annotation instability alone.

TABLE VI
REPRESENTATIVE FIXED-SPLIT FALSE-POSITIVE AND FALSE-NEGATIVE EXAMPLES SELECTED FROM THE AGGREGATED ERROR FILE, WITH BRIEF ENGLISH GLOSSES FOR READABILITY.

False Positive		False Negative	
Example	Label	Example	Label
ບໍ່ ok <i>not OK</i>	0	ເປັນຍັງຄື ທັງຫມົດທີ່ຈຶ່ງບໍ່ໄດ້ <i>why does nothing work?</i>	1
ປ່ຽນໂທລະສັບໃໝ່ <i>changed to a new phone</i>	0	ຄັກຢູ່ ແຕ່ໃຫ້ອັບເດດຄູໂພດ <i>it is good, but please update it</i>	1
ເບີລະຫັດwhatsapp <i>WhatsApp verification code</i>	0	ແຕ່ໃຫ້ປັບປຸງໃຫ້ໄວກ່ວາເກົ່າ <i>please make it faster</i>	1
bug ຫລາຍ <i>too many bugs</i>	0	ໂຄ້ດດີ ຖ້າບໍ່ອັບເດດແອປ ແຮງຊີດີ <i>the code is good, but the update hurts the app</i>	
ແອັບຊ້າສຸດ <i>the app is very slow</i>	0	ຂອບເຂດສິ່ງ ກ້ວາງໄກລ ສິ່ງໄວ ບໍ່ມີບັນຫາ <i>wide service area, fast delivery, no problems</i>	

Figure 4 provides a complementary phrase-level view of the same error set using the 30 most frequent short false-positive and false-negative samples. This visualization is intended as a qualitative summary of recurring failure expressions rather than as a replacement for the quantitative counts in Figure 3.



Fig. 4. Phrase-level word clouds built from the 30 most frequent short false-positive and false-negative samples in the fixed-split experiments. Larger phrases indicate more frequently repeated failure cases.

5.2.4. Practical Implications

From a deployment perspective, the results support a tiered selection strategy. When predictive quality is the main priority, XLM-RoBERTa is the most reliable choice. When computational efficiency matters, LoRA offers a better trade-off than retraining the full model. Although this study does not report dedicated inference latency, the much smaller trainable footprint of LoRA suggests lower adaptation and deployment overhead than full fine-tuning in resource-constrained settings. When infrastructure or maintenance constraints dominate, SVM remains a credible baseline because it is stable, interpretable, and inexpensive to train. More broadly, the benchmark suggests that practical Lao sentiment systems should be selected not only by raw accuracy, but also by how well they balance language fit, efficiency, and operational simplicity.

TABLE VII
EFFICIENCY COMPARISON BETWEEN FULL FINE-TUNING AND LoRA.

Model	Strategy	Trainable params	Trainable ratio	Avg. epoch time (s)	Total train time (s)	F1-macro
XLM-RoBERTa	Full Fine-tuning	278,045,186	1.0000	267.31	6682.66	0.9617
XLM-RoBERTa	LoRA	1,476,866	0.0053	117.80	2945.07	0.9589
mBERT	Full Fine-tuning	177,854,978	1.0000	205.82	5145.48	0.6057
mBERT	LoRA	886,274	0.0050	116.92	2922.95	0.6048
mBERT-Lao	Full Fine-tuning	124,647,170	1.0000	177.52	4438.10	0.9328
mBERT-Lao	LoRA	1,476,866	0.0117	113.74	2843.53	0.9235

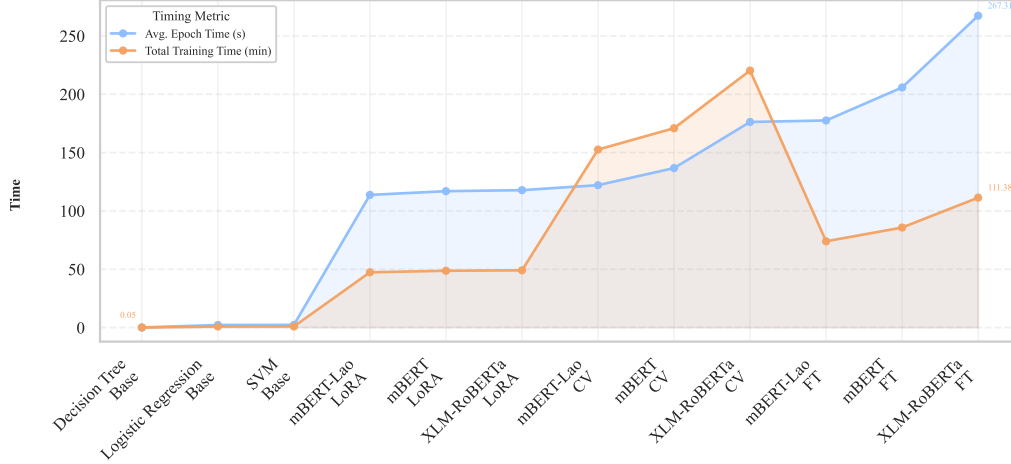


Fig. 5. Timing comparison across PLM settings and classical baselines, sorted by average epoch time.

6 Conclusion

In this study, we established a comprehensive benchmark for Lao sentiment classification, utilizing a large-scale, filtered dataset of real-world mobile application reviews. Our systematic evaluation of classical baselines and advanced Pre-trained Language Models reveals that XLM-RoBERTa delivers the most robust performance, while Lao-specific pretraining significantly enhances cross-lingual transfer in under-resourced settings. Furthermore, we demonstrated that Parameter-Efficient Fine-Tuning (PEFT) via LoRA achieves a practical balance between predictive quality and computational efficiency, making it a viable strategy for resource-constrained deployments. The persistent competitiveness of traditional SVM baselines further underscores the importance of localized lexical features in short, informal Lao feedback.

Despite these advancements, the study identifies critical challenges posed by linguistic ambiguity, noise, and semantic shifts in code-mixed environments. Future research should prioritize enhancing dataset diversity and label consistency, while exploring more granular analysis frameworks, such as aspect-level sentiment classification, to provide deeper insights into user experience. By releasing our dataset and trained models, we aim to support the development of more scalable and accurate NLP tools for the Lao digital ecosystem.

References

- [1] K. L. Tan, C. P. Lee, and K. M. Lim, "A survey of sentiment analysis: Approaches, datasets, and future research," *Applied Sciences*, vol. 13, no. 7, p. 4550, 2023.
- [2] K. Alahmadi, S. Alharbi, J. Chen, and X. Wang, "Generalizing sentiment analysis: a review of progress, challenges, and emerging directions," *Social Network Analysis and Mining*, vol. 15, no. 1, p. 45, 2025.
- [3] N. Lin, Y. Fu, C. Chen, Z. Yang, and S. Jiang, "Laoplms: Pre-trained language models for lao," in *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, 2022, pp. 6506–6512.
- [4] H. T. Nguyen, T. Palongve, C. Q. Nguyen, and K. T. Le, "Semi-automatic construction and benchmarking of a word-segmented corpus for lao using llms and transformer models," *Informatica*, vol. 49, no. 27, 2025.
- [5] M. Subramanian, K. Shanmugavadivel *et al.*, "Kec_tech_titans@ dravidianlangtech 2025: Sentiment analysis for low-resource languages: Insights from tamil and tulu using deep learning and machine learning models," in *Proceedings of the Fifth Workshop on Speech, Vision, and Language Technologies for Dravidian Languages*, 2025, pp. 278–282.

- [6] M. Chowdhury, A. Laskar, T. Ahmad, and A. T. Wasi, "Mysticciol@ dravidianlangtech 2025: A hybrid framework for sentiment analysis in tamil and tulu using fine-tuned sbert embeddings and custom mlp architectures," in *Proceedings of the Fifth Workshop on Speech, Vision, and Language Technologies for Dravidian Languages*, 2025, pp. 167–172.
- [7] T. Durairaj, B. R. Chakravarthi, A. Hegde, H. L. Shashirekha, R. Natarajan, S. Thavareesan, R. Sakuntharaj, C. Rajkumar, P. Shetty, H. S. Kumar *et al.*, "Overview of the shared task on sentiment analysis in tamil and tulu," in *Proceedings of the Fifth Workshop on Speech, Vision, and Language Technologies for Dravidian Languages*, 2025, pp. 732–738.
- [8] W. Zhang, X. Li, Y. Deng, L. Bing, and W. Lam, "A survey on aspect-based sentiment analysis: Tasks, methods, and challenges," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 11, pp. 11 019–11 038, 2022.
- [9] S. U. S. Chebolu, F. Dernoncourt, N. Lipka, and T. Solorio, "A review of datasets for aspect-based sentiment analysis," in *Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2023, pp. 611–628.
- [10] J. Mir, A. Mahmood, S. Khatoon, S. Hussain, S. S. Ullah, and J. Iqbal, "Application domains of aspect and sentiment classification techniques: A survey," *Neurocomputing*, vol. 622, p. 129237, 2025.
- [11] J. Šmíd and P. Kral, "Cross-lingual aspect-based sentiment analysis: A survey on tasks, approaches, and challenges," *Information Fusion*, vol. 120, p. 103073, 2025.
- [12] C. Wu, B. Ma, Y. Liu, Z. Zhang, N. Deng, Y. Li, B. Chen, Y. Zhang, Y. Xue, and B. Plank, "M-absa: A multilingual dataset for aspect-based sentiment analysis," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 2025, pp. 2530–2557.
- [13] N. Z. Lamin and A. A. Aziz, "Cross-lingual sentiment analysis in low-resource languages: A recent review on tasks, methods and challenges," *International Journal of Advanced Computer Science & Applications*, vol. 16, no. 11, 2025.
- [14] S. Ruder, N. Constant, J. Botha, A. Siddhant, O. Firat, J. Fu, P. Liu, J. Hu, D. Garrette, G. Neubig *et al.*, "Xtreme-r: Towards more challenging and nuanced multilingual evaluation," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 2021, pp. 10 215–10 245.
- [15] X. Han, Z. Zhang, N. Ding, Y. Gu, X. Liu, Y. Huo, J. Qiu, Y. Yao, A. Zhang, L. Zhang *et al.*, "Pre-trained models: Past, present and future," *AI open*, vol. 2, pp. 225–250, 2021.
- [16] J. Zhang, X. Qiu, T. Shen, W. Wang, K. Zhang, and W. Feng, "Cross-lingual word alignment for asean languages with contrastive learning," in *2024 International Conference on Asian Language Processing (IALP)*. IEEE, 2024, pp. 61–66.
- [17] B. Min, H. Ross, E. Sulem, A. P. B. Veyseh, T. H. Nguyen, O. Sainz, E. Agirre, I. Heintz, and D. Roth, "Recent advances in natural language processing via large pre-trained language models: A survey," *ACM Computing Surveys*, vol. 56, no. 2, pp. 1–40, 2023.
- [18] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "Lora: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022.
- [19] Y. Mao, Y. Ge, Y. Fan, W. Xu, Y. Mi, Z. Hu, and Y. Gao, "A survey on lora of large language models," *Frontiers of Computer Science*, vol. 19, no. 7, p. 197605, 2025.
- [20] S. Nwaiwu, "Parameter-efficient fine-tuning for low-resource text classification: a comparative study of lora, ia3, and reft," *Frontiers in big Data*, vol. 8, p. 1677331, 2025.
- [21] J. Opitz, "A closer look at classification evaluation metrics and a critical reflection of common evaluation practice," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 820–836, 2024.